Mini Data Indexer and Query Tool

Complexity Analysis & Design Report

This report documents the data structures, algorithmic complexity, and design decisions behind the Mini Data Indexer and Query Tool a pure Python system for loading, indexing, and querying large CSV datasets without external libraries. Each query module implements a naive baseline alongside an optimised strategy so their performance can be directly compared.

Made with GAMMA

1. Dataset Overview

1.1 Schema

Field	Type	Range / Values
record_id	int	1 000 001 – 1 101 001 (mostly unique)
name	str	480 combinations (24 first × 20 last names)
category	str	electronics, books, clothing, furniture, sports, toys, food, automotive
region	str	Marmara, Aegean, Central, Black Sea, Mediterranean, Eastern, Southeastern
year	int	2015 – 2024
score	int	1 – 100
value	float	100.00 – 9999.00

Made with GAMMA

1.2 Dataset Sizes

File	Records	Seed	Duplicate Rate	Purpose
dataset_small.csv	2 000	42	~2%	Development & quick tests
dataset_main.csv	100 000	7202	~2%	Benchmarking & performance

Fixing the random seed guarantees that every run with the same arguments produces an identical dataset essential for reproducible benchmarks.

Made with GAMMA

2. Data Loading — loader.py

The loader converts raw CSV rows into typed **Record** objects. Using `__slots__` instead of a plain dict eliminates the per-instance `__dict__`, reducing memory consumption by approximately 35% at 100 000+ rows while keeping attribute access clean and explicit.

Design Choice	Reason
`__slots__` on Record	~35% memory saving — no per-object `__dict__`
Typed fields (int/float/str)	Catches bad data at load time, not at query time
Returns List[Record]	Simple, indexable; callers build their own indices
Skips bad rows (log & continue)	Resilience: one bad row never aborts the load

Time complexity: $O(n)$ | Space complexity: $O(n)$

Made with GAMMA

3. ID Lookup — Step 3

Two strategies are implemented for locating a record by its **record_id**.

Strategy	Build Cost	Query Cost	Space	When to Use
Linear Search	—	$O(n)$	$O(1)$	One-off lookup, tiny dataset
Hash Map (dict)	$O(n)$	$O(1)$	$O(n)$	Any scenario with multiple queries

With 1 000 queries on 100 000 records, linear search performs ~100 000 000 comparisons. The hash map performs 100 000 (to build) plus 1 000 (one per query) = ~101 000 operations total a theoretical speedup of roughly 990 $\times$, confirmed empirically at ~1 900 $\times$ in benchmarks.

Made with GAMMA

4. Frequency Count — Step 4

A single $O(n)$ pass over the records builds a **dict** mapping each categorical value to its count. Python's dict provides $O(1)$ average-case insert and lookup, so the entire frequency table is built in one linear scan. Sorting the k distinct values for display costs $O(k \log k)$ — negligible since k is at most 8 (categories) or 7 (regions).

Total complexity: $O(n + k \log k) \approx O(n)$ | Space: $O(k)$

Made with GAMMA

5. Duplicate Detection — Step 5

Duplicate rule: two records share the same **(name, year)** pair. The naive approach compares every pair of records $O(n^2)$ operations which is approximately 10 billion comparisons at $n = 100,000$. The optimised approach uses a dict keyed by (name, year):

Pass	Action
For each record	Compute key = (record.name, record.year)
If key in seen dict	Record is a duplicate — append pair to results
Else	Store record in seen dict and continue

Time: $O(n)$ | Space: $O(n)$ | Speedup over naive: $O(n^2) \rightarrow O(n)$

Made with GAMMA

6. Top-K Query — Step 6

Returns the k highest-scoring records. Two strategies are compared:

Strategy	Time Complexity	Space	$n = 100k, k = 10$
Sort-based	$O(n \log n)$	$O(n)$	~1 700 000 ops
Heap-based	$O(n \log k)$	$O(k)$	~330 000 ops

The heap strategy keeps a min-heap of exactly k elements. For each new record, if its score exceeds the heap minimum, it replaces it with one $O(\log k)$ heap operation. The sort strategy always processes all n elements at $O(\log n)$ cost each — wasteful when $k \ll n$. At $n = 100,000$ and $k = 10$ the heap does $\sim 5\times$ less work, and the gap widens as n grows.

Made with GAMMA

7. Range Query — Step 7

Returns all records where **value** falls within $[low, high]$. Two strategies are compared:

Strategy	Build Cost	Query Cost	100 queries, $n = 100k$
Linear Scan	—	$O(n)$	~350 ms (measured)
Binary Search	$O(n \log n)$	$O(\log n + k)$	~16 ms (measured)

The binary search strategy sorts the dataset by value once ($O(n \log n)$), then uses **bisect_left** and **bisect_right** from Python's *bisect* module to locate the inclusive slice $[low, high]$ in $O(\log n)$. The one-time sort cost amortises quickly: 100 range queries cost $O(n \log n + 100 \log n) = O(n \log n)$ total, versus $O(100n)$ for linear scan.

Made with GAMMA

8. Benchmark Results (n = 100 000)

All experiments were run on the 100 000-record dataset. Timings use *time.perf_counter()* for high-resolution measurement.

Experiment	Method	Time (ms)	Speedup
ID Lookup (1000 queries)	Linear search	~800 ms	baseline
ID Lookup (1000 queries)	Hash map	~0.4 ms	**~1900×**
Top-10	Sort-based	~18 ms	baseline
Top-10	Heap-based	~6 ms	**~3×**
Top-100	Sort-based	~16 ms	baseline
Top-100	Heap-based	~7 ms	**~2.4×**
Range Query (100 queries)	Linear scan	~350 ms	baseline
Range Query (100 queries)	Binary search	~16 ms	**~22×**
Duplicate Detection	Hash dict	~77 ms	$O(n)$ vs $O(n^2)$
Frequency Count (cat)	Hash dict	~16 ms	$O(n)$

The largest speedup is in ID lookup: the hash map is nearly 2 000 times faster than linear search across 1 000 queries. Range query shows a 22× speedup — after the one-time sort, binary search locates any range in microseconds regardless of dataset size. The heap advantage grows with n ; at $k = 10$ it is already 3× faster than full sort.

9. Design Principles

Principle	Implementation
Separation of concerns	generator / loader / queries / experiments / main — one job per file
No external libraries	Only Python stdlib: csv, bisect, heapq, time, argparse
Reproducibility	Fixed random seeds guarantee identical datasets across runs
OOP with clean interfaces	Classes with documented public methods; no global state
Resilience	Bad CSV rows are logged and skipped; loading never aborts
Type hints throughout	All functions annotated; catches bugs early in development

10. Correctness Verification

Each optimised strategy is verified to produce identical results to its naive counterpart:

Query	Verification Method
ID Lookup	hash map result == linear search result for every test ID
Top-K	sort-based and heap-based return the same score set (order may differ for ties)
Range Query	binary search and linear scan return the same record count and record set
Duplicates	all (name, year) pairs in result set verified to have count >= 2 in raw data
Frequency	sum of all counts == total number of loaded records

Made with GAMMA